Complete Search (Tam Arama)

Tüm çözümleri düşünerek yapılan aramadır. (Brute force)

S ={1,2,3,4,5} kümesinin tüm alt kümelerini listeleyen bir algoritma yazınız.

```
#include <iostream>
#include <bits/stdc++.h>


using namespace std;

int main()
{
    int a[5] = {1,2,3,4,5};
    // bu kümenin tüm alt kümelerini bulalım
    int n=5;
    for(int i=0;i < (1<<n); i++) // 2^n
    {
        vector<int> subset;
        cout <<" \n";
        for(int z=0;z<sizeof(a)/sizeof(a[0]);z++)
            if(i&1<<z) subset.push_back(a[z]); //örneğin i=25 için 11001 1,4 ve 5'i listeye eklemek lazım
            // bunun için 11001 ile z=4 için 1<<z 16 yani 1000 & işlemiyle true olduğundan 4 ünci elemanı vector'e ekliyorum.
        for(int j=0;j<subset.size();j++)
            cout<<subset[j]<<" ";
    }

}
```

Complete Search (Tam Arama)

Uva 725

abcde rakamları birbirinden farklı bir sayıdır.
fghij rakamları birbirinden farklı bir sayıdır.
İlk basamak 0 olabilir.
abcde/fghij = N sayısı bir tam sayıdır.
$2 \leq N \leq 79\ dur$

Olası tüm abcde, fghij ikililerini bulunuz.

Örnek
abcde 79546
fghij 01283 olabilir. 79546/1283 = 62'dir.

Yöntem:
fghij en küçük 01234
fghij en büyük 98765 olabilir.
Tüm olası fghij değerleri için fghij *N değeri hesaplanır, basamakların farklı olduğu durumlar bulunur.

```
int main(){
    int abcde, fghij;
        for(int N=2;N<=79;N++) {
        for(fghij=1234; fghij<=98765; fghij++){
            abcde = N * fghij;

            if(abcde > 98765)
              break;

           if(checkValues(abcde, fghij)){
             printf("%d  %d %d \n",abcde, fghij, N);
            }
        }
        }
    }
```

```
bool checkValues(int a, int b){
    set<int> s;

     if(a<10000)
     s.insert(0);
    if(b<10000)
     s.insert(0);

    while(a != 0){
          s.insert(a%10);
          a/=10;
    }

    while(b != 0){
          s.insert(b%10);
          b/=10;
    }

    if(s.size() != 10)
          return false;
     if(s.size() == 10)
          return true;
}
```

Budama Yöntemi (Pruning)

Çözümü daha nasıl hızlı hale getirebiliriz?

Complete Search (Tam Arama)

Permütasyon bulma

```
#include <iostream>

#include <bits/stdc++.h>

using namespace std;


void display(vector<int> a, int n){
    for(int i=0;i<n;i++) cout << a[i] << " ";
    cout << endl;
}

int main()
{
    //Dizinin uzunluğunu gir
    int n;
    cin >> n;

    //vectorü tanımla
    vector<int> a(n);

    //Kümenin elemanlarını gir
    for(int i=0; i<n; i++){
        cin >> a[i];
    }

    do{
        //Permütasyonu yazdır
        display(a, n);
    }while(next_permutation(a.begin(), a.end())); //Bir sonraki permütasyona geç

    return 0;
}
```

Binary Search


```
for (int i = 0; i < n; i++) {
    if (array[i] == x) {
        // x found at index i
    }
}
```

O(n)

```
int a = 0, b = n-1;
while (a <= b) {
    int k = (a+b)/2;
    if (array[k] == x) {
        // x found at index k
    }
    if (array[k] > x) b = k-1;
    else a = k+1;
}
```

O(log n)

Coin Problem

$\{1, 2, 5, 10, 20, 50, 100, 200\}$

Yukarıdaki paralarla n=520 €'yi oluşturacak minimum sayıda parayı bulunuz.

Cevap : 2*200+1*100+1*20 yani **4**

Nasıl Bulunur?

Paraların arasında 2 katından azından bir para olmadığı için en büyük para ile başlayarak çözüm bulunabilir.
Çünkü 2 tane 100 yerine 1 tane 200 koymak her zaman daha az parayı buldurur.
Kodu kontrol et

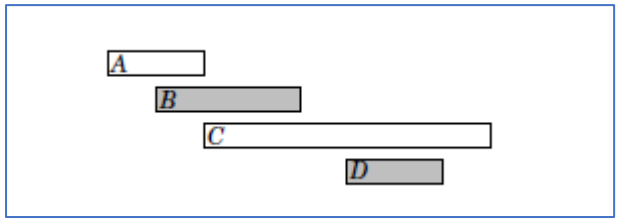
Örneğin
{1,2,5,10,20,50,100,160,200} şeklinde paralar olsaydı cevap ne olurdu?

Çizelgeleme Problemi

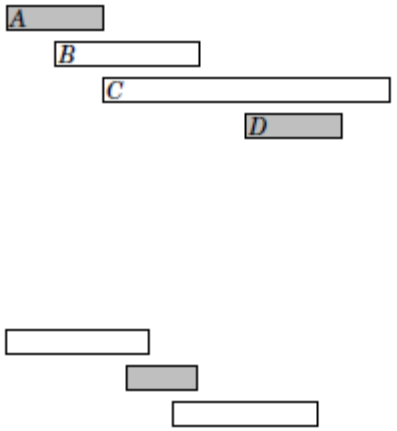
event	starting time	ending time
A	1	3
B	2	5
C	3	9
D	6	8

Aynı anda 2 iş yapılamadığına göre yapılabilecek en fazla kaç iş vardır?

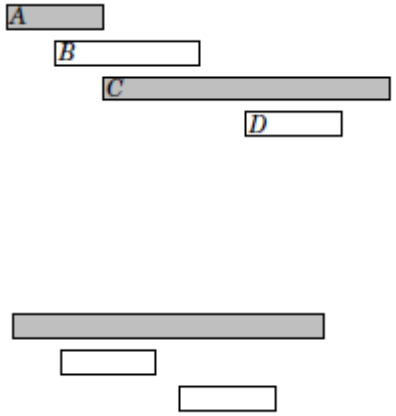
Cevap: 2 (A ve D, B ve D, A ve C olabilir)



Alg 1: En kısa olanı seç



Alg 2: En erken başlayabileceğini seç



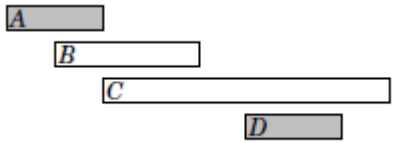
Greedy Algoritmalar (Açgözlü Algoritmalar)

Ders 3

Çizelgeleme Problemi

event	starting time	ending time
A	1	3
B	2	5
C	3	9
D	6	8

Alg 3:En erkeni seç



OK

```
void printMaxActivities(int s[], int f[], int n)
{
    int i, j;

    cout <<"Seçilen işler "<< endl;

    // Bitim zamanı olacağı için ilk işi alıyoruz.
    i = 0;
    cout <<" "<< i;

    // Kalan işler için
    for (j = 1; j < n; j++)
    {

      if (s[j] >= f[i])
      {
          cout <<" " << j;
          i = j;
      }
    }
}

// driver program to test above function
int main()
{
    //işlerin başlama zamanları
    int s[] =  {1, 3, 0, 5, 8, 5};
    //işlerin bitiş zamanları(sıralı)
    int f[] =  {2, 4, 6, 7, 9, 9};
    int n = sizeof(s)/sizeof(s[0]);
    printMaxActivities(s, f, n);
    return 0;
}
```

Minimizing Sum Problemi

$$|a_1 - x| + |a_2 - x| + \cdots + |a_n - x|.$$

$[1, 2, 9, 2, 6]$

Toplamı minimize eden x sayısını bulunuz.

$[1, 2, 2, 6, 9]$

Bu dizinin medyanı toplamı minimize eder.

Yani x=2 dir.

$$(a_1 - x)^2 + (a_2 - x)^2 + \cdots + (a_n - x)^2.$$

$[1, 2, 9, 2, 6]$

Toplamı minimize eden x sayısını bulunuz.

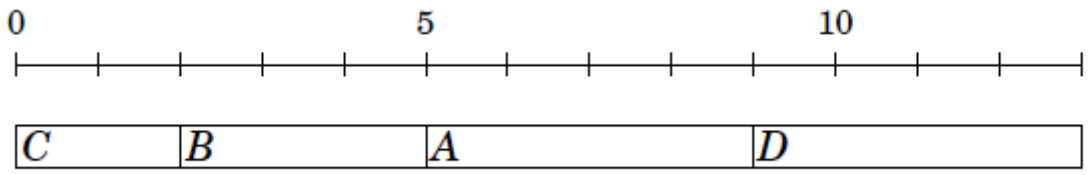
Bu dizinin ortalaması toplamı minimize eder.

Yani x=4 olur.

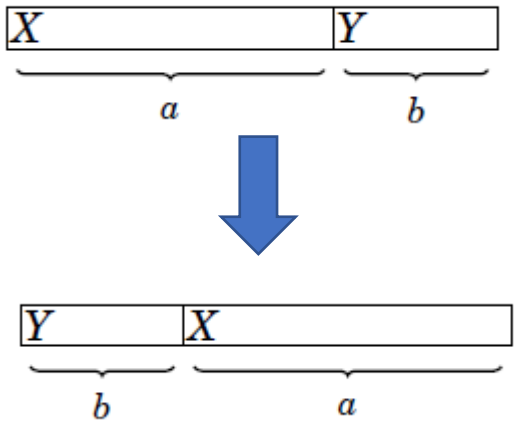
Task Problemi

task	duration	deadline
A	4	2
B	3	5
C	2	7
D	4	5

Bir işin puanı termin tarihi ile işin bitimi arasındaki arasındaki fark kadarsa, toplam puanı nasıl maksimize ederiz?



İşleri en kısadan büyüğe göre sıralamak toplamı maksimize eder.



C'den 5 puan
B'den 0 puan
A'dan -7 puan
D'den -8 puan
Toplam -10 puan

Ödev

UVA 11242

A racing bicycle is driven by a chain connecting two sprockets. Sprockets are grouped into two clusters: the front cluster (typically consisting of 2 or 3 sprockets) and the rear cluster (typically consisting of between 5 and 10 sprockets). At any time the chain connects one of the front sprockets to one of the rear sprockets.

The drive ratio – the ratio of the angular velocity of the pedals to that of the wheels – is the quotient n/m where n is the number of teeth on the rear sprocket and m is the number of teeth on the front sprocket. Two drive ratios $d_1 < d_2$ are adjacent if there is no other drive ratio $d_1 < d_3 < d_2$. The *spread* between a pair of drive ratios $d_1 < d_2$ is their quotient: d_2/d_1.

You are to compute the maximum spread between two adjacent drive ratios achieved by a particular pair of front and rear clusters.

Input

Input consists of several test cases, followed by a line containing 0. Each test case is specified by the following input:

- f: the number of sprockets in the front cluster;
- r: the number of sprockets in the rear cluster;
- f integers, each giving the number of teeth on one of the gears in the front cluster;
- r integers, each giving the number of teeth on one of the gears in the rear cluster.

You may assume that no cluster has more than 10 sprockets and that no gear has fewer than 10 or more than 100 teeth.

Output

For each test case, output the maximum spread rounded to two decimal places.

Sample Input

```
2 4
40 50
12 14 16 19
0
```

Sample Output

```
1.19
```

Ödev

UVA 10670

Ders 3

Paperwork is beginning to pile up on your desk, and tensions at the workplace are starting to mount. Your boss has threatened to fire you if you don't make any progress by the end of the day. You currently have N units of paperwork on your desk, and your boss demands that you have exactly M units of paperwork left by the end of the day.

The only hope for you now is to hire help. There are various agencies which offer paperwork reduction plans:

- For $\$A$ they will reduce your paperwork by one unit.
- For $\$B$ they will reduce your entire paperwork by half (rounding down when necessary).

Note that work can never be reduced to less than 0.

Your task now is to produce a sorted table of agency names and their respective minimum costs to solve your workload problem.

Input

The first line of input consists of a single positive integer representing the number of cases to follow. Each case begins with three positive integers separated by spaces:

N - your starting workload, M - your target workload, and L - the number of work reduction agencies available to you, ($1 \le M \le N \le 100000$, $1 \le L \le 100$). The next L lines have the format '*agency name*:A,B', where A and B are the rates as described above for the given agency. ($0 \le A, B \le 10000$). The length of the agency name will be between 1 and 16, and will consist only of capital letters. Agency names will be unique.

Output

For each test case, print '`Case` X', with X being the case number, on a single line, followed by the table of agency names and their respective minimum costs, sorted in non-decreasing order of minimum costs. Sort job agencies with identical minimum costs in alphabetical order by agency name. For each line of the table, print out the agency name, followed by a space, followed by the minimum required cost for that agency to solve your problem.

Sample Input

```
2
100 5 3
A:1,10
B:2,5
C:3,1
1123 1122 5
B:50,300
A:1,1000
C:10,10
D:1,50
E:0,0
```

Sample Output

```
Case 1
C 7
B 22
A 37
Case 2
E 0
A 1
D 1
C 10
B 50
```